

UNIVERSIDAD CATÓLICA BOLIVIANA
“SAN PABLO”
SEDE TARIJA

MANUAL DE LABORATORIO
Circuitos Electrónicos II (IMT-221)

Análisis Analógico y Validación mediante Ecosistema Python-First

Docente: M.Sc. Ing. Bernardo Quiroga Turdera
Carrera: Ingeniería Mecatrónica

Índice

1. LAB 1: Caracterización Dinámica de Impedancia y Puente de Wheatstone en CA	3
1.1. Configuración de Esquema e Instrucciones en LTSpice	3
1.2. Código Completo en Python (lab1_wheatstone_ac.py)	3
2. LAB 2: Filtro de Muesca Pasivo (Twin-T Notch) a 50 Hz	5
2.1. Configuración de Esquema e Instrucciones en LTSpice	5
2.2. Código Completo en Python (lab2_twin_t_notch.py)	5
3. LAB 3: Amplificador de Instrumentación Discreto de 3 Op-Amps	6
3.1. Configuración de Esquema e Instrucciones en LTSpice	6
3.2. Código Completo en Python (lab3_inamp_calibration.py)	6
4. LAB 4: Limitaciones Dinámicas en CA (Slew Rate y GBW)	7
4.1. Configuración de Esquema e Instrucciones en LTSpice	7
4.2. Código Completo en Python (lab4_slew_rate_gbw.py)	7
5. LAB 5: Rectificador Activo de Precisión (Superdiodo)	9
5.1. Configuración de Esquema e Instrucciones en LTSpice	9
5.2. Código Completo en Python (lab5_superdiode_evaluator.py)	9
6. LAB 6: Filtro Activo Pasa-Bajos de 4.º Orden Butterworth	10
6.1. Configuración de Esquema e Instrucciones en LTSpice	10
6.2. Código Completo en Python (lab6_butterworth_4th.py)	10
7. LAB 7: Estabilidad y Compensación de Fase In-Loop	11
7.1. Configuración de Esquema e Instrucciones en LTSpice	11
7.2. Código Completo en Python (lab7_phase_margin_estimator.py)	11
8. LAB 8: Etapa de Potencia Clase AB y Disipación Térmica	12
8.1. Configuración de Esquema e Instrucciones en LTSpice	12
8.2. Código Completo en Python (lab8_thermal_regression.py)	12

1. LAB 1: Caracterización Dinámica de Impedancia y Puente de Wheatstone en CA

1.1 Configuración de Esquema e Instrucciones en LTSpice

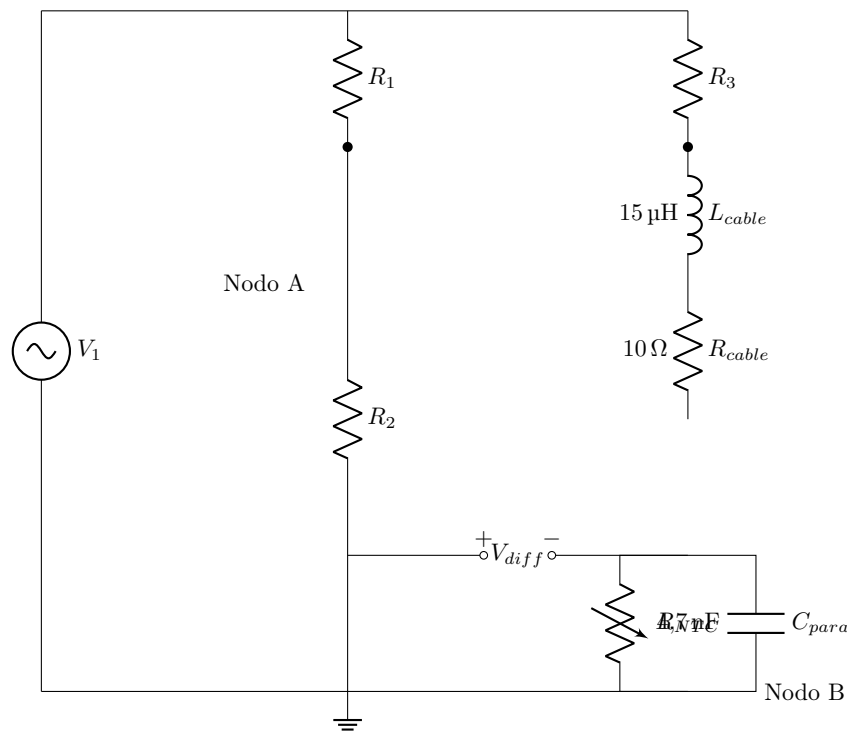
Instrucciones y Comandos SPICE

Componentes y Conexiones:

- **Fuente de alimentación (V_1):** Fuente de CA colocada entre el nodo de entrada y tierra. Propiedades: **SINE(0 1 10k)** (Amplitud pico 1 V, 10 kHz). En *AC Amplitude* colocar 1.
- **Ramas fijas del puente:** $R_1 = 10\text{ k}\Omega$ (V_{in} a Nodo A), $R_2 = 10\text{ k}\Omega$ (Nodo A a GND), $R_3 = 10\text{ k}\Omega$ (V_{in} a Nodo B).
- **Rama del sensor:** Resistor $R_{cable} = 10\text{ }\Omega$ en serie con inductor $L_{cable} = 15\text{ }\mu\text{H}$. A continuación, en serie, el paralelo de R_{NTC} con el capacitor parásito $C_{para} = 4,7\text{ nF}$.

Comandos SPICE:

- Análisis temporal: **.tran 0.5m** (Simula 500 μs , equivalente a 5 ciclos de 10 kHz).
- Barrido de temperatura: **.step param RNTC list 12.5k 10k 8.05k 6.5k 5.3k**
- Medición: Graficar señal diferencial **V(nodeA)-V(nodeB)**.



1.2 Código Completo en Python (lab1_wheatstone_ac.py)

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 # 1. Carga de datos exportados desde el osciloscopio (CSV)
6 try:
7     df = pd.read_csv("lab1_medicion_30C.csv")
8     t, v_in, v_b = df['Time'].values, df['Channel1'].values, df['Channel2'].values
9 except FileNotFoundError:
10     t = np.linspace(0, 0.0005, 1000)
11     v_in = 1.0 * np.cos(2 * np.pi * 10000 * t)
12     v_b = 0.5 * np.cos(2 * np.pi * 10000 * t - np.radians(162.4))
13
14 v_a = v_in / 2.0
15 v_out_exp = v_a - v_b
16
17 # 2. Extraccion de amplitud pico y desfase temporal

```

```

18 v_out_pico_exp = np.max(v_out_exp)
19 idx_pico_in = np.argmax(v_in[:200])
20 idx_pico_out = np.argmax(v_out_exp[:200])
21 delta_t_exp = t[idx_pico_out] - t[idx_pico_in]
22 fase_grados_exp = delta_t_exp * 10000.0 * 360.0
23
24 # 3. Modelo Teorico Analitico
25 T_kelvin = 30.0 + 273.15
26 R_0, Beta, T_0 = 10000.0, 3950.0, 298.15
27 R_ntc = R_0 * np.exp(Beta * (1.0 / T_kelvin - 1.0 / T_0))
28
29 omega = 2 * np.pi * 10000.0
30 Z_c = 1.0 / (1j * omega * 4.7e-9)
31 Z_l = 1j * omega * 15e-6
32 Z_paralelo = (R_ntc * Z_c) / (R_ntc + Z_c)
33 Z_sensor = 10.0 + Z_l + Z_paralelo
34
35 V_out_teorico = 1.0 * (Z_sensor / (10000.0 + Z_sensor) - 0.5)
36 v_out_pico_teorico = np.abs(V_out_teorico)
37 fase_grados_teorica = np.angle(V_out_teorico, deg=True)
38
39 # 4. Metricas de Error y Graficacion
40 err_amp = np.abs(v_out_pico_exp - v_out_pico_teorico) / v_out_pico_teorico * 100.0
41 err_fase = np.abs(fase_grados_exp - fase_grados_teorica) / np.abs(fase_grados_teorica) * 100.0
42
43 print(f"Amplitud Teorica:  {v_out_pico_teorico:.4f} V | Exp: {v_out_pico_exp:.4f} V | Error: {err_amp:.2f}%"
44       )
45 print(f"Fase Teorica:      {fase_grados_teorica:.2f} deg | Exp: {fase_grados_exp:.2f} deg | Error: {err_fase:.2f}%"
46       )
47
48 plt.figure(figsize=(9, 4.5))
49 plt.plot(t * 1e6, v_in, 'k--', label="$V_{in}$ Portadora (10 kHz)", alpha=0.5)
50 plt.plot(t * 1e6, v_out_exp, 'r-', label="$V_{out}$ Diferencial Medida")
51 plt.xlabel("Tiempo ($\mu$ s)")
52 plt.ylabel("Tension (V)")
53 plt.title("Laboratorio 1: Respuesta Diferencial del Puente a 30 C")
54 plt.grid(True)
55 plt.legend()
56 plt.tight_layout()
57 plt.savefig("lab1_resultado.png")
58 plt.show()

```

2. LAB 2: Filtro de Muesca Pasivo (Twin-T Notch) a 50 Hz

2.1 Configuración de Esquema e Instrucciones en LTSpice

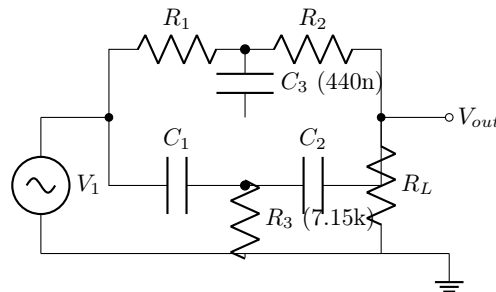
Instrucciones y Comandos SPICE

Componentes y Conexiones:

- **Fuente (V_1):** Conectada a la entrada. Propiedad: **AC 1**.
- **Rama Superior (Paso-Bajo T):** Dos resistores $R_1 = R_2 = 14,3\text{ k}\Omega$ en serie. Entre su nodo central y GND, colocar un capacitor $C_3 = 440\text{ nF}$ ($2C$, formado por dos de 220 nF en paralelo).
- **Rama Inferior (Paso-Alto T):** Dos capacitores $C_1 = C_2 = 220\text{ nF}$ en serie. Entre su nodo central y GND, colocar un resistor $R_3 = 7,15\text{ k}\Omega$ ($R/2$, formado por dos de $14,3\text{ k}\Omega$ en paralelo).
- **Carga (R_L):** Conectada entre la salida unificada y GND.

Comandos SPICE:

- **.ac dec 1000 1 1k** (Barrido en frecuencia logarítmico).
- **.step param RL list 100k 10k 1k** (Evaluación del efecto de carga).



2.2 Código Completo en Python (lab2_twin_t_notch.py)

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 try:
6     data = pd.read_csv("lab2_notch_experimental.csv")
7     freq, v_in, v_out = data['Freq'].values, data['Vin'].values, data['Vout'].values
8     gain_db_exp = 20 * np.log10(v_out / v_in)
9 except FileNotFoundError:
10    freq = np.logspace(0.5, 2.5, 50)
11    w = 2 * np.pi * freq
12    w0 = 2 * np.pi * 50.0
13    H_s = ((1j*w)**2 + w0**2) / ((1j*w)**2 + 4*(1j*w)*w0*(1 + 14300/10000) + w0**2)
14    gain_db_exp = 20 * np.log10(np.abs(H_s)) + np.random.normal(0, 0.5, len(freq))
15
16 R, C, R_L = 14300.0, 220e-9, 10000.0
17 w0_teorico = 1.0 / (R * C)
18 f0_teorico = w0_teorico / (2 * np.pi)
19
20 f_vec = np.logspace(0, 3, 1000)
21 s = 2j * np.pi * f_vec
22 H_teorico_carga = (s**2 + w0_teorico**2) / (s**2 + 4 * s * w0_teorico * (1 + R / R_L) + w0_teorico**2)
23 gain_db_teorico = 20 * np.log10(np.abs(H_teorico_carga))
24
25 idx_min_exp = np.argmin(gain_db_exp)
26 print(f"Frecuencia Notch Teorica: {f0_teorico:.2f} Hz")
27 print(f"Frecuencia Notch Exp: {freq[idx_min_exp]:.2f} Hz")
28 print(f"Atenuacion a f0 Exp: {gain_db_exp[idx_min_exp]:.2f} dB")
29
30 plt.figure(figsize=(9, 4.5))
31 plt.semilogx(f_vec, gain_db_teorico, 'b-', label="Modelo Analitico ($R_L = 10\text{ k}\Omega$)")
32 plt.semilogx(freq, gain_db_exp, 'ro', label="Medicion Experimental")
33 plt.axvline(50.0, color='k', linestyle=':', label="Frecuencia Red (50 Hz)")
34 plt.xlabel("Frecuencia (Hz)")
35 plt.ylabel("Ganancia (dB)")
36 plt.title("Laboratorio 2: Respuesta Frecuencial del Filtro Twin-T Notch")
37 plt.grid(True, which="both")
38 plt.legend()
39 plt.tight_layout()
40 plt.savefig("lab2_resultado.png")
41 plt.show()

```

3. LAB 3: Amplificador de Instrumentación Discreto de 3 Op-Amps

3.1 Configuración de Esquema e Instrucciones en LTSpice

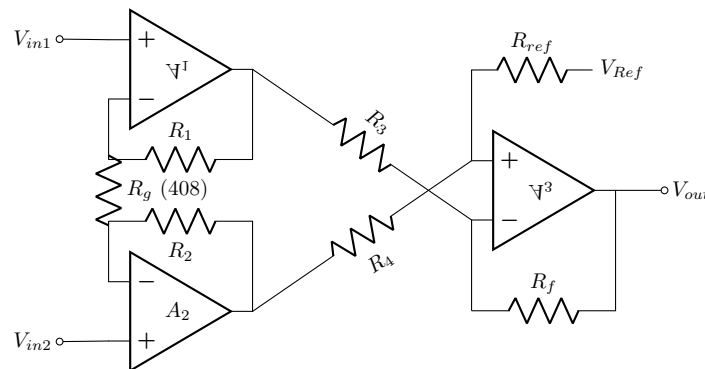
Instrucciones y Comandos SPICE

Componentes y Conexiones:

- **Etapas Buffer de Entrada (A_1, A_2):** A_1 : Entrada no inversora a V_{in1} . $R_1 = 10\text{ k}\Omega$ entre su entrada inversora y su salida. A_2 : Entrada no inversora a V_{in2} . $R_2 = 10\text{ k}\Omega$ entre su entrada inversora y su salida. Resistor $R_g = 408\text{ }\Omega$ entre los nodos inversores de A_1 y A_2 .
- **Etapas Diferencial de Salida (A_3):** Resistores $R_3 = R_4 = 10\text{ k}\Omega$ desde las salidas de A_1 y A_2 hacia las entradas inversora y no inversora de A_3 . Resistor $R_f = 10\text{ k}\Omega$ entre la salida de A_3 y su entrada inversora. $R_{ref} = 10\text{ k}\Omega$ desde la entrada no inversora de A_3 hacia V_{Ref} .

Comandos SPICE:

- Para curva CC: `.dc Vdiff -0.05 0.05 0.001` (Rampa diferencial).
- Para CMRR: `.ac dec 100 1 100k` con tolerancia `.param tol=0.05`.



3.2 Código Completo en Python (lab3_inamp_calibration.py)

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from scipy.stats import linregress
5
6 try:
7     data = pd.read_csv("lab3_inamp_calib.csv")
8     v_diff = data['V_diff_mv'].values / 1000.0 # Convertir a Volts
9     v_out = data['V_out_volts'].values
10 except FileNotFoundError:
11     v_diff = np.linspace(-0.05, 0.05, 21)
12     v_out = 50.12 * v_diff + 0.015 + np.random.normal(0, 0.002, len(v_diff))
13
14 reg = linregress(v_diff, v_out)
15 slope, intercept, r_squared, reg.rvalue**2
16
17 v_out_lineal = slope * v_diff + intercept
18 inl_absoluto = np.abs(v_out - v_out_lineal)
19 inl_porcentaje = (np.max(inl_absoluto) / 5.0) * 100.0 # Respecto a escala 5V
20
21 print(f"Ganancia Diferencial Medida (A_d): {slope:.3f}")
22 print(f"Offset Residual en Salida: {intercept * 1000.0:.2f} mV")
23 print(f"Coefficiente Determinacion R2: {r_squared:.6f}")
24 print(f"Error de No-Linealidad (INL): {inl_porcentaje:.4f} %")
25
26 plt.figure(figsize=(8, 4.5))
27 plt.plot(v_diff * 1000.0, v_out, 'ro', label="Mediciones Experimentales")
28 plt.plot(v_diff * 1000.0, v_out_lineal, 'b-', label=f"Ajuste Lineal ($A_d={slope:.2f}$)")
29 plt.xlabel("Tension Diferencial de Entrada $V_{diff}$ (mV)")
30 plt.ylabel("Tension de Salida Acondicionada $V_{out}$ (V)")
31 plt.title("Laboratorio 3: Curva de Calibracion Estatica del In-Amp")
32 plt.grid(True)
33 plt.legend()
34 plt.tight_layout()
35 plt.savefig("lab3_resultado.png")
36 plt.show()
```

4. LAB 4: Limitaciones Dinámicas en CA (Slew Rate y GBW)

4.1 Configuración de Esquema e Instrucciones en LTSpice

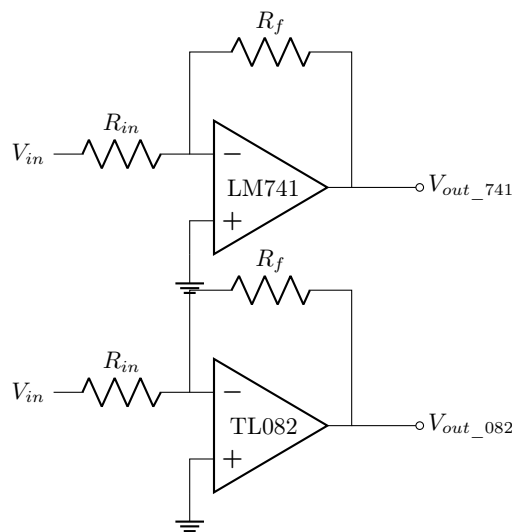
Instrucciones y Comandos SPICE

Componentes y Conexiones:

- Dos esquemas idénticos en paralelo para comparar el LM741 frente al TL082.
- **Configuración Inversora** ($A_v = -10$): Resistor de entrada $R_{in} = 10\text{ k}\Omega$. Resistor de realimentación $R_f = 100\text{ k}\Omega$.

Comandos SPICE:

- **Prueba SR (Gran Señal):** Fuente `PULSE(-0.4 0.4 0 10n 10n 50u 100u)` (800 mV_{pp}). Comando: `.tran 100u`.
- **Prueba GBW (Pequeña Señal):** Fuente `AC 0.01`. Comando: `.ac dec 100 1k 10M`.



4.2 Código Completo en Python (lab4_slew_rate_gbw.py)

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 try:
6     df = pd.read_csv("lab4_slew_rate_data.csv")
7     t, v_741, v_082 = df['Time'].values, df['V_741'].values, df['V_082'].values
8 except FileNotFoundError:
9     t = np.linspace(0, 100e-6, 1000)
10    v_741 = np.clip(10 * (t - 10e-6) * 0.5e6 - 4, -4, 4)
11    v_082 = np.clip(10 * (t - 10e-6) * 13e6 - 4, -4, 4)
12
13 # Derivada numerica discreta para Slew Rate (dv/dt)
14 dt = np.diff(t)
15 dv_741 = np.diff(v_741) / dt
16 dv_082 = np.diff(v_082) / dt
17
18 sr_741_v_us = np.max(np.abs(dv_741)) / 1e6
19 sr_082_v_us = np.max(np.abs(dv_082)) / 1e6
20
21 print(f"Slew Rate Medido LM741: {sr_741_v_us:.2f} V/us (Nominal ~ 0.5 V/us)")
22 print(f"Slew Rate Medido TL082: {sr_082_v_us:.2f} V/us (Nominal ~ 13.0 V/us)")
23
24 fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(9, 6), sharex=True)
25 ax1.plot(t * 1e6, v_741, 'r-', label=f"LM741 (SR = {sr_741_v_us:.2f} V/μs)")
26 ax1.plot(t * 1e6, v_082, 'b-', label=f"TL082 (SR = {sr_082_v_us:.2f} V/μs)")
27 ax1.set_ylabel("Tension $V_{out}$ (V)")
28 ax1.set_title("Laboratorio 4: Limitacion Dinamica por Slew Rate")
29 ax1.grid(True)
30 ax1.legend()
31
32 ax2.plot(t[: -1] * 1e6, dv_741 / 1e6, 'r--')
33 ax2.plot(t[: -1] * 1e6, dv_082 / 1e6, 'b--')

```

```
34 ax2.set_xlabel("Tiempo ( $\mu$  s)")
35 ax2.set_ylabel("Derivada  $dv/dt$  (V/ $\mu$  s)")
36 ax2.grid(True)
37 plt.tight_layout()
38 plt.savefig("lab4_resultado.png")
39 plt.show()
```


5. LAB 5: Rectificador Activo de Precisión (Superdiodo)

5.1 Configuración de Esquema e Instrucciones en LTSpice

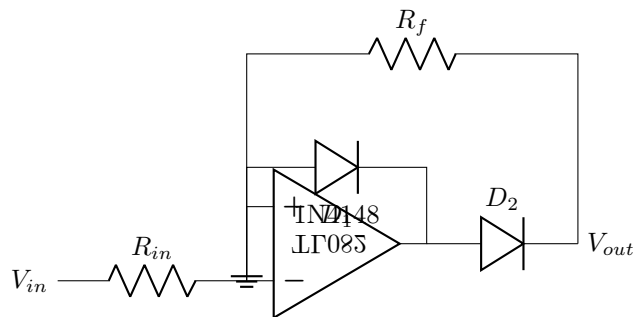
Instrucciones y Comandos SPICE

Componentes y Conexiones:

- Op-Amp TL082. Resistor de entrada $R_{in} = 10\text{ k}\Omega$ conectado al nodo inversor.
- Diodo de lazo D_1 (1N4148) con ánodo en el nodo inversor y cátodo en la salida del Op-Amp.
- Diodo de salida D_2 (1N4148) con ánodo en la salida del Op-Amp y cátodo hacia el nodo V_{out} .
- Resistor de realimentación $R_f = 10\text{ k}\Omega$ conectado entre V_{out} y el nodo inversor.

Comandos SPICE:

- Baja frecuencia: `SINE(0 0.1 100)` (100 mV pico, 100 Hz), `.tran 20m`.
- Alta frecuencia: `SINE(0 0.1 10k)` (100 mV pico, 10 kHz), `.tran 0.2m`.



5.2 Código Completo en Python (lab5_superdiode_evaluator.py)

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from scipy.integrate import simpson
5
6 try:
7     df = pd.read_csv("lab5_superdiode_data.csv")
8     t, v_in, v_out = df['Time'].values, df['Vin'].values, df['Vout'].values
9 except FileNotFoundError:
10     t = np.linspace(0, 0.02, 1000)
11     v_in = 0.1 * np.sin(2 * np.pi * 100 * t)
12     v_out = np.where(v_in < 0, -v_in, 0) # Rectificación inversora ideal
13
14 v_out_ideal = np.where(v_in < 0, -v_in, 0.0)
15
16 # Cálculo del Error Integral (Regla de Simpson)
17 error_absoluto = np.abs(v_out - v_out_ideal)
18 error_integral = simpson(error_absoluto, x=t)
19 print(f"Error Integral de Rectificación: {error_integral * 1e6:.4f} uV-s")
20
21 plt.figure(figsize=(9, 4.5))
22 plt.subplot(1, 2, 1)
23 plt.plot(t * 1000, v_in, 'k--', label="$V_{in}$ (100 mVp)")
24 plt.plot(t * 1000, v_out, 'g-', label="$V_{out}$ Rectificada")
25 plt.xlabel("Tiempo (ms)")
26 plt.ylabel("Tensión (V)")
27 plt.title("Ondas Temporales")
28 plt.grid(True)
29 plt.legend()
30
31 plt.subplot(1, 2, 2)
32 plt.plot(v_in * 1000, v_out * 1000, 'm.', label="Curva XY Medida")
33 plt.plot(v_in * 1000, v_out_ideal * 1000, 'k--', label="Transferencia Ideal")
34 plt.xlabel("Entrada $V_{in}$ (mV)")
35 plt.ylabel("Salida $V_{out}$ (mV)")
36 plt.title("Curva de Transferencia")
37 plt.grid(True)
38 plt.legend()
39 plt.tight_layout()
40 plt.savefig("lab5_resultado.png")
41 plt.show()

```

6. LAB 6: Filtro Activo Pasa-Bajos de 4.º Orden Butterworth

6.1 Configuración de Esquema e Instrucciones en LTSpice

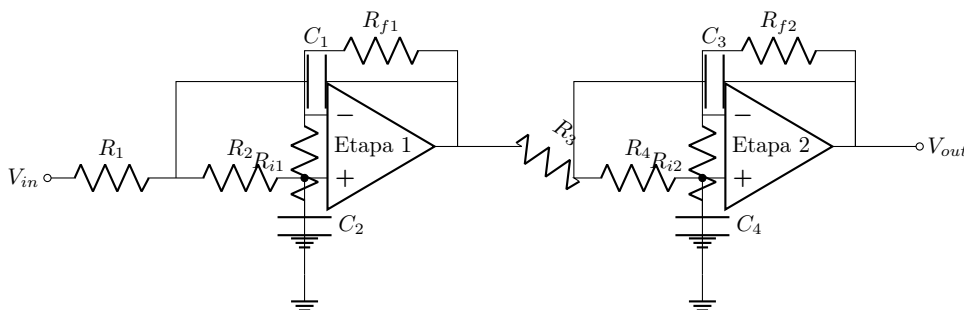
Instrucciones y Comandos SPICE

Componentes y Conexiones (Cascada Sallen-Key):

- **Etapa 1** ($Q_1 = 1,3065$): $R_1 = R_2 = 15,8\text{ k}\Omega$, $C_1 = C_2 = 1\text{ }\mu\text{F}$. Op-Amp inversor auxiliar de ganancia $A_{v1} = 2,234$ ($R_{i1} = 10\text{ k}\Omega$, $R_{f1} = 12,3\text{ k}\Omega$).
- **Etapa 2** ($Q_2 = 0,5412$): $R_3 = R_4 = 15,8\text{ k}\Omega$, $C_3 = C_4 = 1\text{ }\mu\text{F}$. Op-Amp inversor auxiliar de ganancia $A_{v2} = 1,152$ ($R_{i2} = 10\text{ k}\Omega$, $R_{f2} = 1,5\text{ k}\Omega$).

Comandos SPICE:

- Análisis AC: `.ac dec 100 0.1 1k`
- Sensibilidad Monte Carlo: `.step param run 1 50` con tolerancias `.param Rval=mc(15.8k, 0.01)`.



6.2 Código Completo en Python (lab6_butterworth_4th.py)

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 try:
6     df = pd.read_csv("lab6_butterworth_bode.csv")
7     freq, v_in, v_out = df['Freq'].values, df['Vin'].values, df['Vout'].values
8     gain_db_exp = 20 * np.log10(v_out / v_in)
9 except FileNotFoundError:
10     freq = np.logspace(-1, 2, 60)
11     fc = 10.0
12     gain_db_exp = 20 * np.log10(2.57 / np.sqrt(1 + (freq / fc)**8)) + np.random.normal(0, 0.3, len(freq))
13
14 # Modelo Teorico
15 f_vec = np.logspace(-1, 2, 500)
16 fc_rad = 2 * np.pi * 10.0
17 s = 2j * np.pi * f_vec
18 H1 = (fc_rad**2) / (s**2 + 0.7654 * fc_rad * s + fc_rad**2)
19 H2 = (fc_rad**2) / (s**2 + 1.8478 * fc_rad * s + fc_rad**2)
20 gain_db_teorica = 20 * np.log10(np.abs(H1 * H2 * 2.234 * 1.152))
21
22 # Estimacion de Pendiente
23 idx_trans = np.where((freq >= 15.0) & (freq <= 40.0))[0]
24 slope_fit = np.polyfit(np.log10(freq[idx_trans]), gain_db_exp[idx_trans], 1)
25 idx_50hz = np.argmin(np.abs(freq - 50.0))
26
27 print(f"Pendiente Medida: {slope_fit[0]:.2f} dB/decada (Teorico: -80)")
28 print(f"Atenuacion a 50 Hz: {gain_db_exp[idx_50hz]:.2f} dB")
29
30 plt.figure(figsize=(9, 4.5))
31 plt.semilogx(f_vec, gain_db_teorica, 'b-', label="Modelo Teorico")
32 plt.semilogx(freq, gain_db_exp, 'ro', label="Mediciones")
33 plt.axvline(10.0, color='g', linestyle='--', label="$f_c = 10\text{ Hz}$")
34 plt.xlabel("Frecuencia (Hz)"); plt.ylabel("Ganancia (dB)")
35 plt.grid(True, which="both")
36 plt.legend()
37 plt.tight_layout()
38 plt.show()

```

7. LAB 7: Estabilidad y Compensación de Fase In-Loop

7.1 Configuración de Esquema e Instrucciones en LTSpice

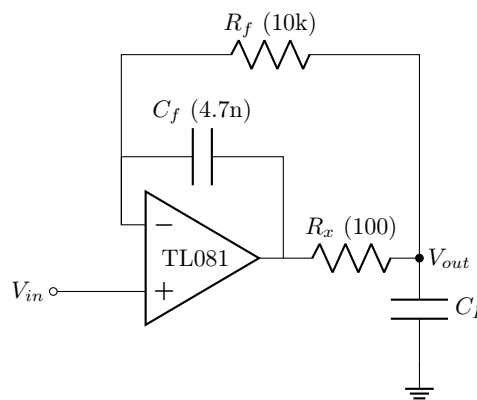
Instrucciones y Comandos SPICE

Componentes y Conexiones:

- **Circuito Inestable:** TL081 seguidor de tensión con $C_L = 47\text{ nF}$ directo a la salida.
- **Circuito Compensado In-Loop:** Resistor $R_x = 100\ \Omega$ entre salida Op-Amp y C_L . Resistor de lazo $R_f = 10\text{ k}\Omega$ desde C_L hacia entrada inversora. Capacitor $C_f = 4,7\text{ nF}$ directo entre salida Op-Amp e inversora.

Comandos SPICE:

- **Respuesta al escalón:** Fuente `PULSE(0 0.5 0 10n 10n 2m 4m)`, `.tran 5m`.
- **Margen de Fase (Middlebrook):** `.ac dec 100 1k 10M`.



7.2 Código Completo en Python (lab7_phase_margin_estimator.py)

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 try:
6     df = pd.read_csv("lab7_step_response.csv")
7     t, v_out = df['Time'].values, df['Vout'].values
8 except FileNotFoundError:
9     t = np.linspace(0, 0.002, 1000)
10    v_out = 0.5 * (1 - np.exp(-0.59 * 31415 * t) * np.cos(31415 * np.sqrt(1 - 0.59**2) * t))
11
12 v_final = np.mean(v_out[-100:])
13 mp_porcentaje = ((np.max(v_out) - v_final) / v_final) * 100.0
14
15 if mp_porcentaje > 0:
16     ln_mp = np.log(mp_porcentaje / 100.0)
17     zeta = np.sqrt(ln_mp**2 / (np.pi**2 + ln_mp**2))
18     pm = zeta * 100.0
19 else:
20     zeta, pm = 1.0, 90.0
21
22 print(f"Sobreimpulso: {mp_porcentaje:.2f} % | Zeta: {zeta:.4f} | PM: {pm:.2f} deg")
23
24 plt.figure(figsize=(9, 4.5))
25 plt.plot(t * 1e3, v_out, 'm-', label=f"Compensada ($M_p$={mp_porcentaje:.1f}\%$)")
26 plt.axhline(v_final, color='k', linestyle='--')
27 plt.xlabel("Tiempo (ms)"); plt.ylabel("Tension (V)")
28 plt.grid(True); plt.legend(); plt.tight_layout(); plt.show()

```

8. LAB 8: Etapa de Potencia Clase AB y Disipación Térmica

8.1 Configuración de Esquema e Instrucciones en LTSpice

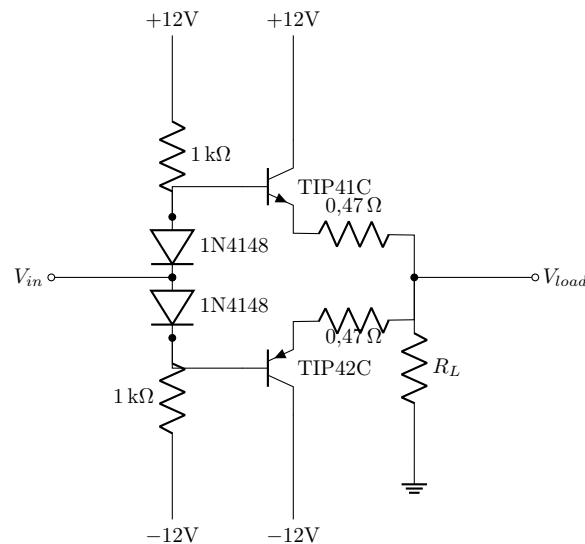
Instrucciones y Comandos SPICE

Componentes y Conexiones:

- Transistores TIP41C (NPN) y TIP42C (PNP). Colectores a rieles +12V y -12V.
- Emisores unidos a V_{load} vía resistores de $0,47\ \Omega$ (1W).
- Carga $R_L = 10\ \Omega$ (10W) entre V_{load} y GND.
- Bases polarizadas mediante dos diodos 1N4148 en serie, con corrientes de reposo fijadas por resistores de $1\text{ k}\Omega$ a los rieles.

Comandos SPICE:

- `.tran 5m` con fuente `SINE(0 8 1k)` (8 V pico).
- **Medición:** Presionar Alt + Click sobre el TIP41C para graficar $I(V_c)*V(c)$.



8.2 Código Completo en Python (lab8_thermal_regression.py)

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from scipy.stats import linregress
5
6 try:
7     df = pd.read_csv("lab8_datos_termicos.csv")
8     i_rms_ma, t_carcasa = df['Corriente_mA'].values, df['Temp_Carcasa_C'].values
9 except FileNotFoundError:
10     i_rms_ma = np.array([100, 200, 300, 400, 500, 600, 700])
11     t_carcasa = np.array([28.1, 32.4, 38.0, 44.2, 51.5, 59.0, 67.2])
12
13 i_rms = i_rms_ma / 1000.0
14 i_peak = i_rms * np.sqrt(2)
15 p_disipada = (12.0 * i_peak / np.pi) - ((i_peak**2 * 10.0) / 4.0)
16
17 reg = linregress(p_disipada, t_carcasa)
18 theta_sa_exp = reg.slope - 0.5
19 t_j_max = t_carcasa[-1] + p_disipada[-1] * 1.67
20
21 print(f"Resistencia Termica (Theta_SA): {theta_sa_exp:.2f} C/W")
22 print(f"Temperatura de Union Maxima: {t_j_max:.2f} C")
23
24 plt.figure(figsize=(8, 4.5))
25 plt.plot(p_disipada, t_carcasa, 'ks', label="Medicion")
26 plt.plot(p_disipada, reg.slope * p_disipada + reg.intercept, 'r-', label=f"Ajuste ({theta_sa_exp:.1f} C/W)")
27 plt.xlabel("Potencia Disipada (W)")
28 plt.ylabel("Temperatura Carcasa (C)")
29 plt.grid(True)

```

```
30 plt.legend()  
31 plt.tight_layout()  
32 plt.show()
```